

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Annexure A

Total Marks:20

MOCK INTERVIEW QUESTIONS

Code of Conduct:

*I_192424078 - Ajay S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

AJAYS

Annexure A

Short Answer Test

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

15. You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using **NumPy and Pandas**? Explain the steps briefly.

1. Cleaning and Standardizing Hospital Patient Data Using NumPy and Pandas

In a hospital, patient data is continuously collected and stored in CSV files from different departments and medical systems. This data may contain missing values, duplicate records, inconsistent units, incorrect data types, and abnormal values. Therefore, a proper data-processing system is required to clean, standardize, and analyze the data efficiently for near real-time reporting. Python libraries such as Pandas and NumPy are well suited for this purpose.

1. Reading and Inspecting the Data

Pandas can be used to read patient data from CSV files using the `read_csv()` function.

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("patient_data.csv")
```

After loading the data, functions such as `head()`, `info()`, and `describe()` can be used to understand the dataset.

```
print(df.head())
print(df.info())
print(df.describe())
```

This helps identify incorrect data types, missing values, and unusual values.

2. Handling Missing Values

Missing values are common in hospital records. Pandas provides functions such as `isnull()`, `fillna()`, and `dropna()` to handle them.

```
print(df.isnull().sum())
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Diagnosis"] = df["Diagnosis"].fillna("Unknown")
```

Numerical missing values can be replaced using the mean or median, while missing categorical values can be replaced with a suitable value such as `"Unknown"`. Critical fields such as Patient ID should generally not be filled arbitrarily and may require the record to be rejected or reviewed.

3. Removing Duplicate Records

Duplicate entries can occur when the same patient information is received multiple times. They can be detected and removed using:

```
df = df.drop_duplicates(
    subset=["Patient_ID", "Timestamp"]
)
```

Using appropriate columns as the duplicate key is important because a patient may legitimately have multiple medical observations.

4. Standardizing Units and Data

Different systems may use different units. For example, temperature may be recorded in Fahrenheit or Celsius. A standard unit such as Celsius can be selected.

```
df["Temperature_C"] = np.where(
    df["Temperature_Unit"] == "F",
    (df["Temperature"] - 32) * 5 / 9,
    df["Temperature"]
)
```

Similarly, weight recorded in pounds can be converted into kilograms:

```
df["Weight_KG"] = np.where(
    df["Weight_Unit"] == "lb",
    df["Weight"] * 0.453592,
    df["Weight"]
)
```

Text values should also be standardized. For example:

```
df["Diagnosis"] = (
    df["Diagnosis"]
    .fillna("Unknown")
    .str.strip()
    .str.title()
)
```

This ensures that values such as "fever", "FEVER" and "Fever " are treated consistently.

5. Indexing and Filtering

Indexing makes it easier to access specific patient records. Patient ID can be used as an index:

```
df.set_index("Patient_ID", inplace=True)
```

A particular patient can then be accessed using:

```
patient = df.loc["P001"]
```

Pandas can also filter records based on conditions. For example, patients with a temperature greater than 38°C can be selected using:

```
high_fever = df[df["Temperature_C"] > 38]
```

Multiple conditions can also be applied:

```
high_risk = df[
    (df["Age"] > 60) &
    (df["Temperature_C"] > 38)
]
```

This is useful for generating reports about high-risk or abnormal patient cases.

6. Aggregation and Reporting

Pandas provides `groupby()` and `agg()` functions for generating summary reports.

For example, the number of patients and average temperature for each diagnosis can be calculated as:

```
summary = df.groupby("Diagnosis").agg(
    Patient_Count=("Age", "count"),
    Average_Age=("Age", "mean"),
    Average_Temperature=("Temperature_C", "mean")
)
```

Such aggregated information can be used by hospital management for daily reports, department-wise analysis, and monitoring patient conditions.

7. Handling Unexpected Data Anomalies

Unexpected values such as a negative age, extremely high temperature, or invalid blood pressure should be detected before reporting.

For example:

```
invalid_age = df[
    (df["Age"] < 0) |
    (df["Age"] > 120)
]
```

Invalid values can be converted to missing values for further processing:

```
df.loc[
    (df["Age"] < 0) |
    (df["Age"] > 120),
    "Age"
] = np.nan
```

Instead of silently deleting anomalous records, they should be stored in a separate error or anomaly file for investigation. This improves data quality and provides traceability.

8. Near Real-Time Processing and Saving

Since hospital data is continuously generated, processing the entire historical dataset every time would be inefficient. New CSV files or newly added records should be processed incrementally. For very large files, Pandas can also process data in chunks:

```
for chunk in pd.read_csv(
    "patient_data.csv",
    chunksize=10000
):
    # Clean and process each chunk
    pass
```

After processing, the cleaned data can be saved into a separate CSV file:

```
df.to_csv(
    "processed_patient_data.csv",
    index=False
)

summary.to_csv(
    "hospital_summary.csv"
```

)

The original raw data should be preserved and should not be overwritten.

Conclusion

Using NumPy and Pandas, a hospital can build an efficient data-processing pipeline for continuously collected patient data. Pandas can handle CSV files, missing values, duplicates, indexing, filtering, aggregation, and data storage, while NumPy can perform efficient numerical calculations, unit conversions, and conditional operations. Anomaly detection and separate error logging ensure that incorrect data does not affect reports.

The overall process can be represented as:

CSV Data → Data Validation → Cleaning → Duplicate Removal → Unit Standardization → Anomaly Detection → Filtering → Aggregation → Processed Data → Hospital Reports

This approach provides accurate, consistent, efficient, and near real-time patient data processing, which is essential for reliable hospital reporting and decision-making.

15. Basic Data Preprocessing Using NumPy and Pandas

Data preprocessing is an important step in machine learning because raw datasets may contain missing values, categorical variables, and numerical features with different scales. Before giving the data to a machine learning model, the data must be cleaned and transformed into a suitable numerical format. NumPy and Pandas provide simple and efficient functions for performing these preprocessing operations.

1. Loading and Understanding the Dataset

The dataset can be loaded using Pandas.

```
import pandas as pd
import numpy as np

df = pd.read_csv("dataset.csv")
```

The structure of the dataset can be checked using:

```
print(df.head())
print(df.info())
print(df.isnull().sum())
```

This helps identify numerical columns, categorical columns, and missing values.

2. Handling Missing Values

Missing values can reduce the accuracy of a machine learning model, so they must be handled before training.

First, missing values can be identified using:

```
print(df.isnull().sum())
```

For numerical columns, missing values can be replaced using the mean or median:

```
df["Age"] = df["Age"].fillna(  
    df["Age"].median()  
)  
  
df["Salary"] = df["Salary"].fillna(  
    df["Salary"].mean()  
)
```

For categorical columns, the missing value can be replaced with the most frequent value or "Unknown":

```
df["City"] = df["City"].fillna("Unknown")
```

If a column contains too many missing values and is not useful, it may also be removed.

3. Encoding Categorical Variables

Machine learning algorithms generally require numerical input, so categorical data such as gender, city, or department must be converted into numerical form.

Label Encoding

For binary or ordered categories, values can be mapped to numbers.

```
df["Gender"] = df["Gender"].map({  
    "Male": 0,  
    "Female": 1  
})
```

For example:

Male → 0
Female → 1

One-Hot Encoding

For categories that do not have an order, one-hot encoding is more suitable. Pandas provides the `get_dummies()` function:

```
df = pd.get_dummies(  
    df,  
    columns=["City", "Department"]  
)
```

For example, a `City` column containing:

Chennai
Delhi
Mumbai

can become:

	City_Chennai	City_Delhi	City_Mumbai
1	0	0	
0	1	0	
0	0	1	

This allows categorical information to be used by a machine learning algorithm.

4. Separating Features and Target

After preprocessing, the input features and target variable should be separated.

For example, if `Purchased` is the target:

```
X = df.drop("Purchased", axis=1)  
y = df["Purchased"]
```

Here, `X` contains the input features and `y` contains the output that the model needs to predict.

5. Normalizing Numerical Features

Numerical features can have very different ranges. For example, age may range from 18–80, while salary may range from 20,000–200,000. Large differences in scale can affect some machine learning algorithms.

One common technique is Min-Max Normalization:

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad =$$

This converts values approximately into the range 0 to 1.

Using NumPy:

```

X_min = X.min()
X_max = X.max()

X_normalized = (
    X - X_min
) / (
    X_max - X_min
)

```

Another method is Standardization:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X = original value
- μ = mean
- σ = standard deviation

Using NumPy:

```

X_standardized = (
    X - X.mean()
) / X.std()

```

Standardization gives features approximately a mean of 0 and a standard deviation of 1.

6. Checking the Preprocessed Data

After preprocessing, the dataset should be checked again.

```

print(X_normalized.head())
print(X_normalized.isnull().sum())

```

The final dataset should contain:

- No unwanted missing values
- Properly encoded categorical variables
- Numerical features on a suitable scale
- Correct data types
- Separate input features and target values

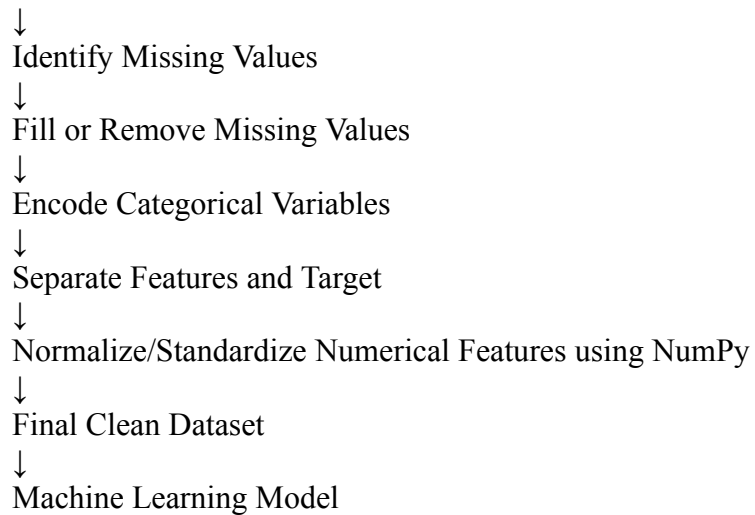
7. Overall Preprocessing Workflow

The complete preprocessing process can be summarized as:

```

Raw Dataset
↓
Load using Pandas

```



Conclusion

NumPy and Pandas provide efficient tools for preparing data for machine learning. Pandas can be used to load the dataset, detect and handle missing values, encode categorical variables, and organize the data, while NumPy can perform efficient numerical operations such as normalization and standardization. Proper preprocessing improves the quality of the dataset and helps machine learning models produce more reliable results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand